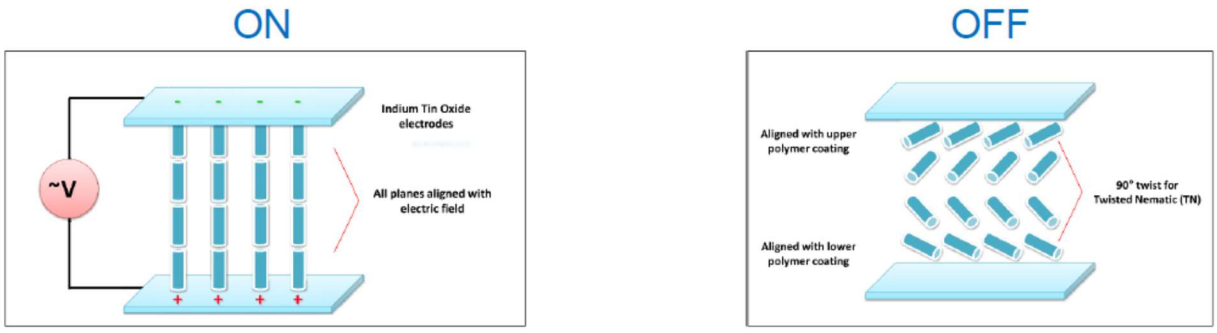
Lab05 - LCD

1. Cos'è un LCD?

LCD significa **Liquid Crystal Display**. Un LCD è un dispositivo che permette di visualizzare informazioni trasformando segnali elettrici in **pixel visibili**.
Un LCD usa dei **cristalli liquidi** per controllare quanta luce passa attraverso lo schermo. I cristalli liquidi sono particolari perché:

- si comportano un po' come un **liquido**, quindi le molecole possono muoversi;
- allo stesso tempo le molecole tendono a disporsi tutte in una certa **direzione**;
- applicando una **tensione elettrica**, possiamo cambiare il loro orientamento.



Immagina ogni segmento come una piccola zona dello schermo che possiamo decidere di mostrare o non mostrare.

- **ON:** applichiamo una tensione abbastanza grande → i cristalli cambiano orientamento → il segmento diventa visibile.
- **OFF:** la tensione non è sufficiente → i cristalli rimangono nel loro orientamento → il segmento non diventa visibile.

La tensione elettrica cambia l'orientamento dei cristalli liquidi, e questo cambia il modo in cui la luce passa attraverso lo schermo. Una cosa importante: **un LCD non produce luce propria** come un LED. Sta semplicemente controllando il passaggio della luce (proveniente dall'ambiente o da una retroilluminazione).

2. LCD 16×2: cosa significa?

16×2 significa:

- 16 caratteri per riga
- 2 righe

Quindi puoi visualizzare al massimo:
1234567890123456
ABCDEFGHIJKLMNOP
Ogni carattere è formato da una matrice di piccoli punti, tipicamente **5×8 pixel**.

3. Come comunica l'STM32 con l'LCD?

L'STM32 deve inviare una sequenza di **comandi e dati** al controller dell'LCD. La comunicazione può avvenire in diversi modi.

Parallelamente

Il metodo classico utilizza diversi GPIO:

STM32 LCD

——— ———

GPIO ——————————> D4

GPIO ——————————> D5

GPIO ——————————> D6

GPIO ——————————> D7

GPIO ——————————> RS

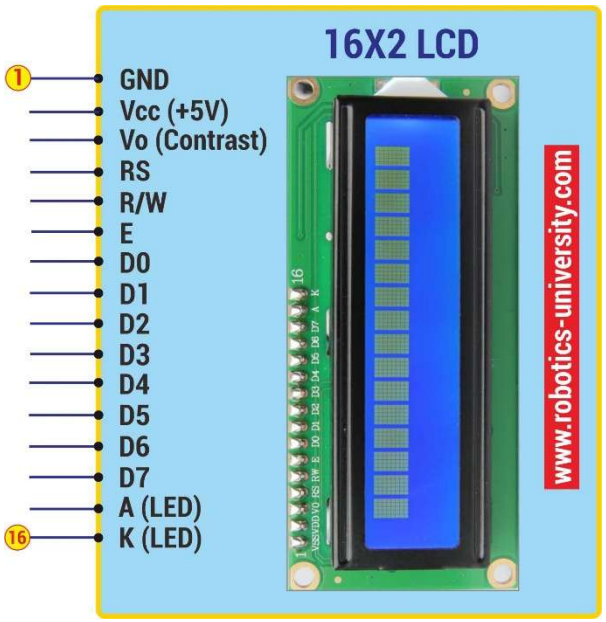
GPIO ——————————> E

Utilizzando solo **D4-D7** si parla di modalità **4-bit**. È molto comune perché permette di ridurre il numero di GPIO necessari.

4. I pin principali dell'LCD

Nel classico LCD HD44780 trovi tipicamente:

Pin	Nome	Funzione
1	VSS	GND
2	VDD	Alimentazione
3	V0	Contrasto
4	RS	Register Select
5	RW	Read/Write
6	E	Enable
7–14	D0–D7	Bus dati
15	A	Backlight +
16	K	Backlight −



5. RS — Register Select

Il pin **RS** dice all'LCD che tipo di informazione stiamo inviando. Ci sono due possibilità:

RS = 0 → comando

Stiamo inviando un'istruzione al controller.
Per esempio:

- Clear Display
- Cursor Home
- Set Cursor
- Display ON/OFF

RS = 1 → dato
Stiamo inviando un carattere da visualizzare.

6. RW — Read/Write

Il pin **RW** stabilisce la direzione della comunicazione.

- RW = 0 → Write
- RW = 1 → Read

Nella maggior parte dei progetti STM32, però, l'LCD viene utilizzato praticamente solo in scrittura. Per questo spesso: RW → GND
e l'STM32 controlla solamente:

- RS
- E
- D4
- D5
- D6
- D7

7. E — Enable

E è il segnale che dice al controller: "Leggi i dati che ti sto mettendo sul bus." In pratica l'STM32 prepara:

- RS
- D4
- D5
- D6
- D7

e poi genera un impulso sul pin **E**.
Quindi una scrittura tipicamente segue:
1. imposto RS
2. imposto D4-D7
3. genero impulso E
4. LCD acquisisce il dato

8. Come entra STM32CubeMX?

Con **STM32CubeMX/CubeIDE** devi innanzitutto configurare i GPIO collegati all'LCD.
Per esempio potresti avere:
LCD STM32
———————————————
RS ———— PA0
E ———— PA1
D4 ———— PA2
D5 ———— PA3
D6 ———— PA4
D7 ———— PA5
In CubeMX imposti questi pin come:
GPIO_Output
Poi CubeMX genera le definizioni e l'inizializzazione GPIO.

A quel punto puoi utilizzare HAL:
HAL_GPIO_WritePin(...)
per mettere HIGH/LOW sui pin.

9. Una tipica libreria LCD

Normalmente **non scrivi tutto questo ogni volta**. Si crea una piccola libreria, ad esempio:

```
LCD_Init();
LCD_Clear();
LCD_SetCursor(0, 0);
LCD_Print("Hello");
```

e internamente queste funzioni utilizzano:

```
HAL_GPIO_WritePin()
HAL_Delay()
```

per comunicare con il controller LCD.
Per esempio:

```
LCD_Init();

LCD_SetCursor(0, 0);
LCD_Print("Hello");

LCD_SetCursor(0, 1);
LCD_Print("STM32");
```

Il risultato sarebbe:

Hello
STM32

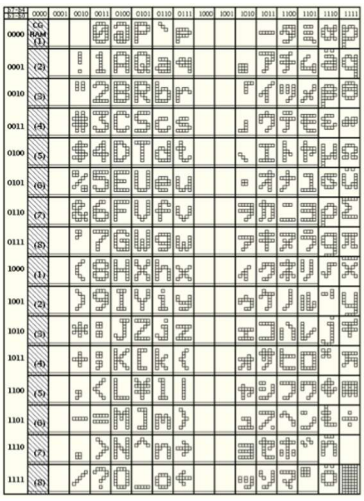
10. Registri Interni

Tre memorie che rappresentano **tre zone diverse della memoria interna di un LCD alfanumerico** (come il classico 16×2).

Memoria	Significato	A cosa serve
CGROM	Character Generator ROM	Contiene i **caratteri già definiti** dal produttore
CGRAM	Character Generator RAM	Contiene i **caratteri personalizzati** creati dall'utente
DDRAM	Display Data RAM	Contiene **quali caratteri devono essere visualizzati e dove**

- ### CGROM — Character Generator ROM

È la memoria che contiene i caratteri **standard** dell'LCD.
Tu dici al display "Visualizza il carattere A" e il controller va nella **CGROM**, trova la forma grafica della A e la visualizza.
ROM → non viene modificata normalmente dall'utente.



- ### CGRAM — Character Generator RAM

Serve invece per creare **caratteri personalizzati**.
Per esempio, potresti voler creare: ❤️
oppure un simbolo particolare che non esiste nella CGROM.
Tu definisci la forma del carattere, e la salvi nella **CGRAM**.
RAM → può essere modificata dall'utente.

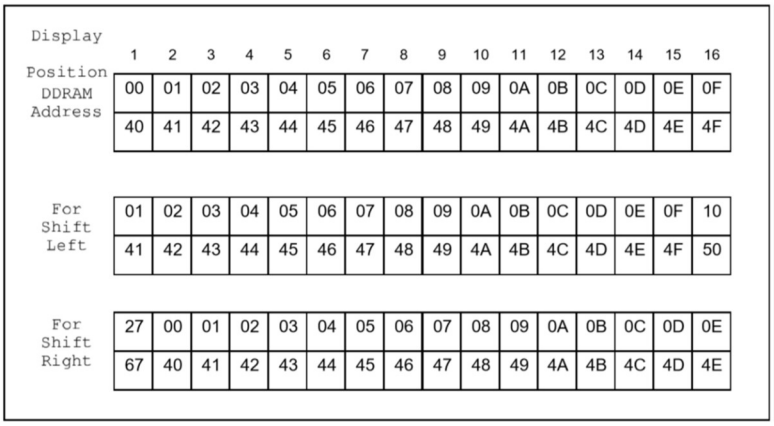
In pratica:

- **CGROM = caratteri già pronti**
- **CGRAM = caratteri che creo io**

- DDRAM — Display Data RAM

La **DDRAM non contiene la forma dei caratteri**, ma contiene **quali caratteri devono essere visualizzati nelle varie posizioni del display**.
Il controller usa poi questi codici per andare a prendere la forma dei caratteri dalla CGROM.
Quindi il processo è:

DDRAM
|
| "A"
▼
CGROM
|
| forma grafica della A
▼
DISPLAY



11. Pin sulla Nucleo

Pin	Funzione
PA4	LCD_BL_ON
PB1	LCD_E
PB2	LCD_RS
PB12	LCD_D4
PB13	LCD_D5
PB14	LCD_D6
PB15	LCD_D7

12. Funzioni

1. lcd_initialize()

void lcd_initialize();

Inizializza il controller dell'LCD, impostandolo in modo che possa essere utilizzato correttamente.
In pratica va chiamata **prima di utilizzare le altre funzioni dell'LCD**.

```
int main(void)
{
    HAL_Init();
    SystemClock_Config();
```

```
    MX_GPIO_Init();

    lcd_initialize();

    // Da questo momento posso usare l'LCD
}
```

2. lcd_backlight_ON()

void lcd_backlight_ON();

Accende la **retroilluminazione (backlight)** dell'LCD. Attenzione: la retroilluminazione è semplicemente la luce dietro al display. **Non è la stessa cosa dell'accensione dei caratteri.**

```
lcd_initialize();

lcd_backlight_ON();
```

3. lcd_backlight_OFF()

void lcd_backlight_OFF();

Spegne la **retroilluminazione** dell'LCD. I caratteri potrebbero comunque rimanere visualizzati, semplicemente senza la luce di background.

4. lcd_println()

void lcd_println(char string[], uint8_t row);

Visualizza una **stringa di testo** su una delle due righe dell'LCD.
Il parametro row indica la riga:

- row = 0 → prima riga
- row = 1 → seconda riga

La lunghezza massima della stringa è **16 caratteri**, perché l'LCD è 16×2.

lcd_println("Hello", 0);

5. lcd_drawBar()

void lcd_drawBar(int value);

Visualizza una **barra grafica** sulla seconda riga dell'LCD.
Il valore può andare da:
0 → 80
Il valore determina quanto sarà lunga la barra.
Ogni incremento del valore abilita una quantità aggiuntiva della **matrice di punti** del display. Quindi è utile, per esempio, per rappresentare:

- una percentuale;

- il livello di un sensore;
- il volume;
- una misura analogica;
- lo stato di avanzamento di un processo.

```
lcd_drawBar(40);
```
Visualizza una barra lunga circa metà del massimo.
Oppure:
```
lcd_drawBar(80);
```
visualizza la barra completa.

6. lcd_clear()

void lcd_clear();

Cancella **tutto ciò che è visualizzato sul display**. Dopo averla chiamata, entrambe le righe risultano vuote.

Funzione	Cosa fa
lcd_initialize()	Inizializza l'LCD
lcd_backlight_ON()	Accende la retroilluminazione
lcd_backlight_OFF()	Spegne la retroilluminazione
lcd_println(string, row)	Scrive una stringa sulla riga 0 o 1
lcd_drawBar(value)	Disegna una barra sulla seconda riga
lcd_clear()	Cancella tutto il display

Un esempio completo potrebbe essere:
```
lcd_initialize();

lcd_backlight_ON();

lcd_println("Temperature:", 0);
lcd_drawBar(40);

HAL_Delay(3000);

lcd_clear();

lcd_println("Done!", 0);
```

Quindi l'idea generale è: **inizializzo → accendo il backlight → scrivo/disegno → eventualmente cancello o aggiorno il display**.